

# Technical audit of the Polygon zkEVM Bridge Service

The `zkevm-bridge-service` is a Go backend `Go Packages` that generates Merkle proofs for cross-chain asset bridging `GitHub` between Ethereum L1 and L2 networks `github` `GitHub` — but **the service is effectively deprecated**, with its parent chain (Polygon zkEVM Mainnet Beta) officially sunsetting `polygon` by mid-2026. The last release (v0.6.3) shipped December 11, 2025, `GitHub` and no commits have appeared in 2026. Despite clean architectural separation and a gRPC-first API design, the codebase shows signs of rapid development with multiple data race fixes, Merkle tree panics, and a single-contributor dependency. `GitHub` Two major security audits (Hexens, Spearbit) found and resolved critical vulnerabilities before the 2023 mainnet beta launch, `Polygon` `Polygon` but centralization risks remain significant — L2BEAT flags the absence of a user exit window `L2BEAT` and the possibility of malicious state finalization. `L2BEAT`

---

## Architecture: five components orchestrating cross-chain proofs

The bridge service sits between the Polygon zkEVM smart contracts and end users (via the Bridge UI or direct API calls). It does not generate zk-proofs itself — its core function is maintaining local copies of Merkle exit trees and producing the sibling-path proofs users need to claim bridged assets on the destination chain. `Go Packages`

**The five core components** work as follows. The **Synchronizer** continuously polls L1 (Ethereum) and multiple L2 networks for `BridgeEvent`, `ClaimEvent`, and `UpdateGlobalExitRoot` emissions from the bridge contracts, writing indexed data to PostgreSQL and handling chain reorganizations. The **Bridge Controller** (`bridgectrl/`) manages one 32-level Keccak256 Merkle tree per network, supporting `AddDeposit()`, `GetProof()`, `ReorgMT()`, and `GetExitRoot()` operations. The **Server** exposes both gRPC and REST (via `grpc-gateway v2`) endpoints — key routes include `/api/v1/bridges/{address}` for deposit queries and `/merkle-proof?deposit_cnt=N&net_id=M` for proof retrieval. `GitHub` The **Claim Transaction Manager** (`claimtxman/`) automates claiming on behalf of users, with retry logic, nonce management, and transaction batching via the `ClaimCompressor` contract for gas optimization. The **Etherman** layer wraps `go-ethereum` to interact with **14+ smart contracts**, including `PolygonZkEVMBridgeV2`, `PolygonZkEVMGlobalExitRootV2`, `PolygonRollupManager`, and sovereign chain variants.

The database layer uses **PostgreSQL** (with SQLite available for development) through `pgx/pgxpool` connection pooling and `sql-migrate` for schema management. Two separate connection pools exist — `SyncDB` for the synchronizer (writer) and `BridgeServer.DB` for the API (reader) `GitHub` — implementing a clean reader/writer separation pattern. Key tables store deposits, claims, exit roots, wrapped token mappings, Merkle tree nodes, and block sync state.

---

## How L1↔L2 bridging actually works

The bridge uses a **three-tier Merkle tree architecture**. Each chain maintains a **Local Exit Tree** (depth 32, append-only). All L2 exit roots aggregate into a **Rollup Exit Tree**. The **Global Exit Root** is computed as `GER = Keccak256(MainnetExitRoot || RollupExitRoot)` and stored in the Global Exit Root Manager on both L1 and L2.

**For L1→L2 deposits**, a user calls `bridgeAsset()` on the L1 bridge contract, which locks tokens `(medium)` and appends an exit leaf containing `(leafType)`, `(originNetwork)`, `(originTokenAddress)`, `(destinationNetwork)`, `(destinationAddress)`, `(amount)`, and `(metadata)`. The updated Mainnet Exit Root propagates to the GER Manager, which recomputes the GER. `(medium)` The L2 sequencer fetches this GER and writes it to the L2 GER Manager `(medium)` at the start of the next batch. The bridge service detects the deposit, indexes it, and generates two Merkle proofs: `(smtProofLocalExitRoot[32])` (proving the leaf exists in the L1 exit tree) and `(smtProofRollupExitRoot[32])` (proving the exit root's inclusion in the rollup tree). `(GitHub)` The user or auto-claim service then calls `claimAsset()` on L2, which verifies both proofs against the on-chain GER `(medium)` and mints wrapped tokens. `(Polygon)` This takes approximately **15 minutes** on testnet.

**For L2→L1 withdrawals**, the flow reverses but adds a critical dependency: the L2 exit root only reaches the L1 GER Manager **after the aggregator generates a zk-proof** for the batch containing the bridge transaction `(Polygon)` and `(verifyBatches())` succeeds on L1. This means L2→L1 claims require batch proving, `(Jarrod Watts)` taking approximately **1 hour** on testnet. The bridge contract's `(verifyLeaf())` function recomputes the leaf hash, walks both proof paths, and confirms `(Keccak256(mainnetExitRoot || rollupExitRoot))` matches a known GER. `(medium)` Double-spending is prevented by a **globalIndex bitmap** — each deposit has a unique index, and once claimed, further claim attempts revert.

---

## Code quality reveals rapid development trade-offs

The repository is **89% Go**, **6.9% Solidity** (contract ABI bindings), and **1.4% PLpgSQL** (database migrations). The directory layout follows clean Go package conventions with well-defined boundaries between `(bridgectrl/)`, `(etherman/)`, `(server/)`, `(synchronizer/)`, `(claimtxman/)`, and `(db/)`. `(GitHub)` The service uses `(urfave/cli/v2)` for CLI management, `(spf13/viper)` for TOML-based configuration, `(go.uber.org/zap)` for structured logging, and `(grpc-ecosystem/grpc-gateway/v2)` for dual-protocol API support.

**Testing infrastructure** includes unit tests with mock interfaces `(make generate-mocks)`, Docker-based E2E tests against real networks (Cardona testnet), benchmarks `(make bench)`, and regression tests with CI automation. The Makefile provides **12+ targets** covering build, test,

lint, Docker, proto generation, and local environment management. (GitHub) CI/CD uses GitHub Actions with (golangci-lint), and releases are automated through GoReleaser with GPG-signed tags.

However, several patterns raise concerns. **Data race issues** required fixes across multiple releases (v0.6.0 PR #776, v0.6.2 PRs #824 and #866), indicating the concurrent access model between the synchronizer and API server was not adequately protected initially. The **Merkle tree** — the cryptographic backbone — suffered a panic bug fixed only in v0.6.3 (#875). (GitHub) The (ClaimTxManager) required a significant refactor in v0.6.2 (#858), and context propagation was corrected late (#864). Integer overflow protections for chain IDs were added retroactively (#697). Schema instability persists, with primary key modifications (#721) and breaking config parameter renames between minor versions ((GenBlockNumber) → (L1GenBlockNumber)). (GitHub) Database credentials are stored as plaintext in TOML config files with no documented secret management integration.

---

## Security posture: audited but centralization-constrained

**Two major external audits** preceded the March 2023 mainnet beta launch. (Polygon) **Hexens** reviewed all 35+ zkEVM components (December 2022 – March 2023), finding 9 vulnerabilities including a critical ERC-777 re-entrancy attack on the bridge contract and missing binary constraint manipulations. All issues were fixed. (Polygon) **Spearbit** conducted three discrete audits covering cryptography, prover, and bridge smart contracts, finding **no critical or high-severity issues in the bridge contracts** specifically — only 3 medium-risk and 16 low-risk findings, all addressed. (Polygon) Certora later audited the Vault Bridge protocol extension in April–May 2025. (Certora) (certora)

**Bug bounty discoveries** have been more alarming. Verichains found a **zkProver proof forgery vulnerability** enabling a malicious Trusted Aggregator to craft valid proofs for arbitrary computations — fixed December 2023. (Verichains) Georg Wiese discovered a **storage machine overflow** allowing false storage reads exploitable for draining token pools. (HackMD) Researcher iczc found a bridge claiming bug preventing L1→L2 asset claims. (TokenInsight) Academic researchers (USENIX Security 2025) uncovered 6 soundness issues including a novel "dual execution path vulnerability" class. (USENIX) The **Immunefi bug bounty** offers up to \$1M for critical zkEVM vulnerabilities (Immunefi) and \$2M for the broader Polygon ecosystem. (Polygon)

The most significant security event was the **March 23, 2024 outage**, when an Ethereum mainnet reorg caused the sequencer to process batches with incorrect timestamps and an invalid global exit root. (Blockworks) The Security Council activated the emergency state for the first time, pausing bridge functionality for ~10 hours. (L2BEAT) (Chainspot) No user funds were lost, but it demonstrated the centralization dependency.

**L2BEAT identifies critical centralization vectors:** no mechanism for users to force transactions if the sequencer censors; only whitelisted proposers can publish state roots (if they fail, withdrawals freeze); contracts are instantly upgradable with Security Council cooperation with **no exit window** for users; (L2BEAT) transaction data is kept off-chain; (L2BEAT) and as of December 2025, pessimistic proofs validate only bridge accounting — **not full L2 state transitions**, meaning a malicious proposer could finalize an invalid state. (L2BEAT) (L2BEAT)

Key management relies on a tiered multisig structure: (L2BEAT) an **8-member Security Council** (6/8 threshold) can activate emergency state, (Polygon) (Polygon) a **5/9 Admin Multisig** (L2BEAT) manages upgrades through a timelock, and **2 whitelisted EOAs** serve as Trusted Aggregator/Proposer.

---

## Dependencies and external integrations

The service integrates with these key external systems:

- **Smart contracts:** (PolygonZkEVMBridgeV2) (L1+L2), (PolygonZkEVMGlobalExitRootV2) (L1), (PolygonZkEVMGlobalExitRootL2) (L2), (PolygonRollupManager) (L1), (ClaimCompressor) (L2), plus sovereign chain variants — **14+ contract bindings** in the (etherman/smartcontracts/) directory
- **Blockchain nodes:** Ethereum L1 via single RPC URL, multiple L2s via URL array (GitHub) (supports Erigon and OP-Geth) (GitHub)
- **Core Go libraries:** (go-ethereum) v1.15.5 (GitHub) (Ethereum interaction), (iden3/go-iden3-crypto) v0.0.17 (Poseidon hashing for ZK-compatible operations), (grpc) v1.67.0, (protobuf) v1.34.1, (viper) v1.19.0, (zap) v1.27.0, (sql-migrate) v1.7.0
- **Database:** PostgreSQL via pgx with connection pooling (MaxConns configurable); optional embedded PostgreSQL in Docker container since v0.6.0 (GitHub)
- **Monitoring:** Prometheus metrics on (/metrics) endpoint (GitHub) (deposit/claim counters, processing latency, DB connection metrics, sync status) (GitHub)

A major milestone in v0.6.0 was **removing the (zkevm-node) dependency** (PR #765), making the bridge service fully standalone. (GitHub) The Go module path still references the original (0xPolygonHermes) organization despite the GitHub transfer to (0xPolygon).

---

## Deployment, configuration, and operations

The service runs via (./bin/zkevm-bridge run --cfg /app/config.toml) (GitHub) and is

configured entirely through **TOML files** loaded by Viper, with environment variable overrides for secrets. Configuration spans seven sections: `(SyncDB)`, `(BridgeServer)`, `(Etherman)` (L1/L2 RPC URLs), `(Synchronizer)` (chunk settings), `(NetworkConfig)` (contract addresses and genesis blocks), `(ClaimTxManager)`, and `(AutoClaimService)`. Multi-L2 support uses array-typed fields (`(L2URLs)`, `(L2GenBlockNumbers)`, `(L2PolygonBridgeAddresses)`, `(RequireSovereignChainSmcs)`). `(GitHub)`

Docker deployment uses a multi-stage Dockerfile with an optional **embedded PostgreSQL** mode `(GitHub)` (v0.6.0-RC15). `(GitHub)` Docker Compose orchestrates the full local stack. GoReleaser handles release automation, and a `(packaging/)` directory supports Debian package builds. Database migrations run automatically via `(sql-migrate)` on startup. Observability comes from zap-structured logging (with configurable levels), Prometheus metrics, and a sync status API endpoint (v0.6.3). `(GitHub)` Version information (`(Version)`, `(GitRev)`, `(GitBranch)`, `(BuildDate)`) is injected at build time `(Go Packages)` via ldflags. `(Go Packages)`

---

## The repository is entering its final chapter

The project shows **unmistakable signs of wind-down**. Polygon Labs CEO Marc Boiron announced the zkEVM Mainnet Beta deprecation on June 11, 2025, `(polygon)` citing \$1M+/year operational losses, `(CryptoPotato)` lack of product-market fit, and the inability to support EIP-4844. `(CCN)` The sequencer will run until approximately June 2026 with forced transactions enabled. `(polygon)` The related `(zkevm-node)` repository was **archived** on February 17, 2025, `(GitHub)` `(GitHub)` and the zkEVM ZK research team spun off into an independent entity (ZisK/SilentSig Switzerland GmbH) on June 13, 2025. `(GitHub)` `(Blocmates)`

Repository metrics confirm this trajectory: **101 stars, 86 forks, 6 open issues, 11 stalled PRs**, and zero commits since December 2025. Development concentrated in a **single contributor** (@ARR552) who authored virtually all v0.6.x releases. The bus factor is critically high. No active security patching has occurred in over 3.5 months.

However, the bridge service retains residual utility. The v0.6.x releases added **sovereign chain support** `(GitHub)` and Agglayer integration, and the README now describes the service as connecting to "the Agglayer" rather than just zkEVM. `(GitHub)` It remains referenced in **Polygon CDK/Kurtosis deployments** for CDK chains. `(GitHub)` Successor infrastructure includes the **Agglayer** (v0.3 launched June 2025, `(Coinfomania)` `(X)` Rust-based at `(github.com/agglayer/agglayer)`, `(GitHub)` using pessimistic proofs via the SP1 zkVM `(GitHub)` for cross-chain settlement `(Medium)` across non-CDK chains. `(L2BEAT)`

## Conclusion

This audit reveals a service that achieved its architectural goals — clean Go package separation, gRPC-first design, multi-L2 Merkle tree management — but accumulated technical debt through

rapid iteration: data races in concurrent access paths, Merkle tree edge-case panics, and schema instability across releases. The security audit history is thorough (Hexens, Spearbit, Certora, Immunefi bug bounty), yet the most consequential risks are structural: **centralization in the sequencer/proposer, absence of a user exit window for contract upgrades, and L2 state validation limited to bridge accounting only** (L2BEAT +2) since the December 2025 migration to pessimistic proofs. (L2BEAT) For teams running CDK chains that still depend on this bridge service, the critical action items are monitoring the Agglayer migration timeline, evaluating the single-maintainer risk, and ensuring the Security Council multisig governance meets their trust assumptions. For the zkEVM Mainnet Beta specifically, the bridge service is in its final operational months. (polygon)